

Common Ownership Replication — QK Collaboration

Backus, Conlon & Sinkinson (2019) | 2013Q3–2025Q4

Does it matter that the same few investors own large stakes in competing companies?

Think about it this way: Vanguard and BlackRock together own 5–10% of nearly every major US company — including direct competitors like United Airlines and Delta Airlines. If the same investor profits when either company does well, that investor has less reason to want them to compete aggressively against each other.

The paper measures this using a number called κ (**kappa**) for every pair of S&P 500 firms. $\kappa = 0$ means the two firms compete normally. $\kappa = 1$ means they behave as if fully merged. The original paper found the average κ tripled from 0.2 (1980) to 0.7 (2017).

We are extending this to 2013–2025 using QUANTkiosk 13(F) holdings data.

1. Setup

```
In [14]: !pip install qkiosk --quiet
```

[notice] A new release of pip is available: 25.3 -> 26.1.1

[notice] To update, run: pip install --upgrade pip

```
In [ ]: import ssl, certifi

def _ssl_context(*args, **kwargs):
    if not args and 'cafile' not in kwargs:
        kwargs['cafile'] = certifi.where()
    return ssl.SSLContext.create_default_context(ssl.SSLContext, *args, *

# The two-line fix that covers all HTTPS libraries on Mac
ssl._create_default_https_context = lambda: ssl.create_default_context(ca

import urllib.request, urllib.error
_ctx = ssl.create_default_context(cafile=certifi.where())
_opener = urllib.request.build_opener(urllib.request.HTTPSHandler(context
urllib.request.install_opener(_opener)

# Quick test
try:
    urllib.request.urlopen("https://www.google.com", timeout=5)
    print("SSL: OK")
except Exception as e:
    print(f"SSL still failing: {e}")

import qkiosk as qk
```

```

import pandas as pd
import numpy as np
import requests

# — API Key —

try:
    from google.colab import userdata
    qk.set_apikey(userdata.get("QK_API_KEY"))
except Exception:
    import os
    if "QK_API_KEY" not in os.environ:
        qk.set_apikey("YOUR_KEY_HERE")

qk.account()

```

SSL: OK

Out[]: QUANTkiosk Account: (Saturday, 16-May-26 00:43:06 UTC)

Daily Quota: 10000 (Hard Quota: 10000)

Daily Usage: 120

Daily quota resets in: 23h 16m

Visit <https://quantkiosk.com/account> to change your plan or explore offerings.

2. How the QK API Works

This section documents what we found by reading the `qkiosk` Python library source code and running empirical measurements across three collection sessions. We include this because it directly motivates the questions in Section 5.

2a. The two ownership endpoints and what they do

The library has two distinct paths for pulling 13F institutional holdings data.

Manager perspective (`qk.institutional`) — *"What does Citadel hold?"*

```

GET /data/ownership?apiKey=...&ids=
{MANAGER_CIK}&yyyy=2013&qq=04
→ Returns all positions held by that manager in that
quarter

```

Issuer perspective (`qk.holders`) — *"Who holds Apple?"*

```

GET /data/instrument?apiKey=...&id=
{FIRM_CIK}&yyyy=2013&qq=04
→ Returns all institutional holders of that firm in that
quarter

```

For our analysis, we need the **issuer perspective**: for every firm in the S&P 500, find all its institutional investors and how many shares they hold. That forms the

ownership matrix we need to compute κ .

2b. Empirical cost measurements

We ran three full collection sessions and measured credits before and after each one.

Session	Credits used	Firms downloaded	Rate	What was pulled
Session 1	480	~24	~20/firm	2013Q4 start + one-time universe cache
Session 2	4,460	302	14.8/firm	2013Q4 (avg 590 investors/firm)
Session 3	10,020	586	17.1/firm	2013Q3 + 2014Q1 (558–629 investors/firm)

The one-time overhead in Session 1 came from loading the QK500 universe. The library makes four separate API calls just to start:

- `qk.univ("QK500")` → ~500 credits
- `univ.to_cik()` → ~821 credits
- `univ.to_name()` → ~821 credits
- `univ.to_ticker()` → ~821 credits

Total startup overhead: **~2,963 credits per script run**. We fixed this by saving the universe to a local JSON file on first run — subsequent runs load it from disk for free.

2c. The architecture difference (why `fn()` is fast and `holders()` is slow)

Reading the library source code revealed the core issue. The fundamentals endpoint and the ownership endpoint use completely different server architectures.

`qk.fn()` — fundamentals (one call, all firms):

```
# From qkiosk/fundamentals.py
POST /data/fundamental
Body: {"ids": "CIK1,CIK2,...,CIK500", "items": "EPS"}

# Server packages all 500 firms → returns 500 pre-signed S3 URLs
# Client downloads all 500 files in parallel
# Total: ~1 API call for 500 firms
```

`qk.holders()` — ownership (one call per firm):

```
# From qkiosk/ownership.py
for cik in all_500_firm_ciks:
    GET /data/instrument?id={cik}&yyyy=2013&qq=04
    # Waits for response, then moves to next firm
# Total: 500 separate API calls
```

At ~15 credits per call, that's 7,500 credits per quarter. For 50 quarters (Q1–Q4, 2013–2025), the total is ~375,000 credits — approximately 38 days of pulling at 10,000/day.

2d. Does cost scale with investor count? (No resolved, the cost remains same even at 2023)

The number of investors per firm has grown substantially over time:

Quarter	Avg investors per firm	Observed rate
2013Q3	558	~17.1 credits/firm
2013Q4	590	14.8 credits/firm
2014Q1	629	~17.1 credits/firm
2023Q4	1,919	untested at scale

quarter investors credits credits_per_investor 2013Q4 1920 20 0.01 2023Q4 4929 20 0.004

Investor count ratio (2023/2013): 2.57x Credit cost ratio (2023/2013): 1.00x → Cost appears flat per call regardless of investor count

2. The Analysis — Computing κ

We're replicating Backus, Conlon & Sinkinson (2019), "*Common Ownership in America: 1980–2017*".

The paper asks: when the same investors (Vanguard, BlackRock) own large stakes in competing firms (United Airlines and Delta), do those firms compete less aggressively?

The key measure is κ_{fg} — how much firm f weights firm g 's profits in its decisions.

Ownership fractions

For firm f and institutional investor s :

$$\beta_{fs} = \frac{\text{shares held by } s \text{ in } f}{\text{total shares of } f \text{ outstanding}}$$

The retail share — the fraction not held by institutions, who exercise no governance:

$$r_f = 1 - \sum_s \beta_{fs}$$

A high retail share amplifies κ : institutional investors control a larger share of governance decisions than their raw ownership percentage suggests.

The profit weight κ

$$\kappa_{fg} = \frac{\sum_s \beta_{fs} \cdot \beta_{gs}}{\sum_s \beta_{fs}^2} \in [0, \infty)$$

Value	Economic meaning
$\kappa = 0$	Normal competition — firm f ignores g
$\kappa = 1$	Merger-equivalent — f values $10f * g$'s profitability of its own
$\kappa > 1$	Tunneling incentive — f would transfer profits to g

The paper found average κ tripled from 0.2 in 1980 to 0.7 in 2017. We extend to 2025.

Geometric decomposition

$$\kappa_{fg} = \underbrace{\cos(\beta_f, \beta_g)}_{\text{investor similarity}} \times \sqrt{\frac{\text{IHHI}_g}{\text{IHHI}_f}}$$

where $\text{IHHI}_f = \sum_s \beta_{fs}^2$ is the investor Herfindahl index.

- **Cosine similarity** rises as index funds push investors toward the same portfolios
- **IHHI ratio**: high retail share $\rightarrow$ low $\text{IHHI}_f \rightarrow$ larger κ_{fg}

3. Live Data Pull

Each call to `/data/instrument` returns every institutional investor that holds shares in one firm for one quarter. This is the raw data that feeds into the κ formula above.

```
In [16]: # Check quota before – we'll compare after to measure exact cost
usage_before = qk.account().usage

aapl = qk.ticker("AAPL")
raw = qk.holders(aapl, yyyyqq=201304).to_df()

usage_after = qk.account().usage
print(f"Credits for this call: {usage_after - usage_before}")
print(f"Rows returned (all share types including options): {len(raw)}")
```

Credits for this call: 20

Rows returned (all share types including options): 2195

```
In [17]: # Filter to equity-only holdings and aggregate sub-managers
equity = (raw
    .query("putCall.isna() and shrsOrPrnAmt > 0", engine="python")
    .groupby("filerName", as_index=False)["shrsOrPrnAmt"].sum()
    .sort_values("shrsOrPrnAmt", ascending=False)
    .assign(shares_M = lambda x: (x["shrsOrPrnAmt"] / 1e6).round(1))
    .reset_index(drop=True))
```

```
)

print(f"Apple Q4 2013 – {len(equity)} institutional investors")
print(f"Total institutional shares: {equity['shrsOrPrnAmt'].sum() / 1e9:.")
print()
equity[["filerName", "shares_M"]].head(10)
```

Apple Q4 2013 – 1917 institutional investors
Total institutional shares: 0.52B

Out[17]:

	filerName	shares_M
0	VANGUARD GROUP INC	44.2
1	State Street Corp	38.2
2	FMR LLC	28.3
3	BlackRock Institutional Trust Company N.A.	23.9
4	Invesco Ltd.	13.3
5	NORTHERN TRUST CORP	13.1
6	PRICE T ROWE ASSOCIATES INC /MD/	10.7
7	BlackRock Fund Advisors	9.3
8	SUSQUEHANNA INTERNATIONAL GROUP LLP	7.8
9	JPMORGAN CHASE & CO	7.6

4. Computing κ — Apple vs Microsoft, Q4 2013

Pull both firms, align the investor vectors, then apply the formula:

$$\kappa_{fg} = \frac{\sum_s \beta_{fs} \cdot \beta_{gs}}{\sum_s \beta_{fs}^2}$$

```
In [18]: # Pull Microsoft for the same quarter
msft_raw = qk.holders(qk.ticker("MSFT"), yyyyqq=201304).to_df()
msft_equity = (msft_raw
               .query("putCall.isna() and shrsOrPrnAmt > 0", engine="python")
               .groupby("filerName", as_index=False)["shrsOrPrnAmt"].sum()
               )

# How many investors do they share?
common = set(equity["filerName"]) & set(msft_equity["filerName"])
print(f"AAPL: {len(equity)} investors | MSFT: {len(msft_equity)} investor
```

AAPL: 1917 investors | MSFT: 1941 investors | Common: 1553

```
In [19]: # Get total shares outstanding from SEC EDGAR XBRL (free, no QK credits)
def get_shares(cik_padded, year):
    r = requests.get(
        f"https://data.sec.gov/api/xbrl/companyfacts/CIK{cik_padded}.json",
        headers={"User-Agent": "research souparneya@gmail.com"},
        verify=certifi.where(), timeout=15
    )
```

```

data = r.json()
from datetime import date
target = date(year, 12, 31)
for ns, field in [
    (data["facts"].get("us-gaap", {}), "CommonStockSharesOutstanding")
    (data["facts"].get("dei", {}), "EntityCommonStockSharesOutstan
]:
    entries = [e for e in ns.get(field, {}).get("units", {}).get("share
        if e.get("form", "") in ("10-K", "10-Q", "10-K/A", "10-Q/A
    if entries:
        closest = min(entries, key=lambda e: abs((date.fromisoformat(
            return closest["val"]
    return None

aapl_total = get_shares("0000320193", 2013)
msft_total = get_shares("0000789019", 2013)

print(f"AAPL shares outstanding (Dec 2013): {aapl_total/1e9:.2f}B")
print(f"MSFT shares outstanding (Dec 2013): {msft_total/1e9:.2f}B")

```

AAPL shares outstanding (Dec 2013): 0.89B
MSFT shares outstanding (Dec 2013): 8.30B

```

In [20]: # Build aligned ownership vectors and compute  $\kappa$ 
all_investors = equity.set_index("filerName")["shrsOrPrnAmt"]
all_msft      = msft_equity.set_index("filerName")["shrsOrPrnAmt"]
universe      = all_investors.index.union(all_msft.index)

beta_aapl = all_investors.reindex(universe, fill_value=0).values / aapl_t
beta_msft = all_msft.reindex(universe, fill_value=0).values / msft_to

kappa_am = np.dot(beta_aapl, beta_msft) / np.dot(beta_aapl, beta_aapl)
kappa_ma = np.dot(beta_msft, beta_aapl) / np.dot(beta_msft, beta_msft)
r_aapl   = 1 - beta_aapl.sum()
r_msft   = 1 - beta_msft.sum()

pd.DataFrame({
    "Firm": ["AAPL", "MSFT"],
    "Retail share (r_f)": [f"{r_aapl:.1%}", f"{r_msft:.1%}"],
    "IHHI": [f"{np.dot(beta_aapl, beta_aapl):.6f}", f"{np.do
    "k toward other firm": [f"{kappa_am:.4f}", f"{kappa_ma:.4f}"],
})

```

```

Out[20]:
   Firm  Retail share (r_f)  IHHI  k toward other firm
0  AAPL                41.5%  0.007887                0.9345
1  MSFT                33.1%  0.008814                0.8362

```

This result is pretty interesting even though it is for one quarter and it may be noise. But when we plot κ over our full time series, we expect to see a clear upward trend as passive index funds grew their stakes through 2013–2025.

The paper finds within-industry κ is especially high, and Apple and Microsoft are broadly both tech firms though they sit in different SIC codes (hardware vs. software), which makes the 0.93 even more notable. The more interesting question for our analysis would be that do firms in completely separate industries show similarly high κ simply because the Big Three own large stakes in both? If a bank and

an airline share the same top five institutional holders, do they also show convergent incentives? That cross-industry test is something the paper touches on but doesn't foreground — and it's where our 2013–2025 data could add something new.

5. Questions

5.1 — Batch endpoint for `/data/instrument`

This is the most crucial question. `/data/fundamental` accepts all firm CIKs in a single POST request — the server packages the data and returns pre-signed S3 download URLs. If `/data/instrument` supports the same pattern, our 38-day pull becomes roughly 3 days.

The cell below sends the batch request. A successful response means batch mode works. Not successful:(

```
In [21]: import os
API_KEY = os.environ.get("QK_API_KEY", "")

batch_response = requests.post(
    "https://api.qkiosk.io/data/instrument",
    json={
        "apiKey": API_KEY,
        "ids":    "0000320193,0000789019,0000019617",    # AAPL, MSFT, JPM
        "yyyy":   "2013",
        "qq":     "04",
    },
    verify=certifi.where(), timeout=20
)

print(f"Status: {batch_response.status_code}")
print(batch_response.text[:600])
```

```
Status: 502
{"message": "Internal server error"}
```

5.2 — Historical universe by date

Today's QK500 reflects the current S&P 500. For 2013, about 45% of the current firms didn't exist yet (Airbnb, Coinbase, etc.). Those slots were held by firms that have since been acquired — Aetna, Celgene, Dell, Time Warner.

The paper uses the **actual historical composition** at each quarter. If QK maintains this, one call resolves everything.

```
In [22]: # Test whether a time-aware universe parameter exists
for call in [
    'qk.univ("QK500", as_of=20131231)',
    'qk.univ("QK500", dt=20131231)',
    'qk.univ("SP500", dt=20131231)',
```



```
]:
    try:
        result = eval(call)
        print(f"✓ {call} → {len(result)} firms returned")
    except TypeError as e:
        print(f"x {call} → unexpected parameter: {e}")
    except Exception as e:
        print(f"x {call} → {type(e).__name__}")
```

```
x qk.univ("QK500", as_of=20131231) → unexpected parameter: univ() got a
n unexpected keyword argument 'as_of'
✓ qk.univ("QK500", dt=20131231) → 500 firms returned
x qk.univ("SP500", dt=20131231) → HTTPError
```

5.3 — CIKs for acquired companies

We reconstructed the historical S&P 500 from Wikipedia's changes log and searched SEC EDGAR for the old CIKs of acquired firms. We found 276 of 333. The remaining 57 are companies whose old SEC records we can't locate through public searches.

To show this works for the ones we did find, the cell below pulls Aetna (AET) — acquired by CVS in 2018, CIK found via EDGAR full-text search.

```
In [23]: # Aetna was in the S&P 500 from our first snapshot (2013) until 2018.
# Its old CIK is 0001122304 — still has historical 13F data.
aetna_url = f"https://api.qkiosk.io/data/instrument?apiKey={API_KEY}&id=0
r = requests.get(aetna_url, verify=certifi.where(), timeout=20)

if r.status_code == 200:
    import io
    aetna = (pd.read_csv(io.StringIO(r.content.decode()))
              .query("putCall.isna() and shrsOrPrnAmt > 0", engine="pyth
              .nlargest(5, "shrsOrPrnAmt")[["filerName", "shrsOrPrnAmt"]]
    aetna["shares_M"] = (aetna["shrsOrPrnAmt"]/1e6).round(1)
    print(f"Aetna (AET) Q4 2013 — top 5 holders:")
    print(aetna[["filerName", "shares_M"]].to_string(index=False))
    print(f"\nThe endpoint works for historical CIKs — we just need to fi
else:
    print(f"HTTP {r.status_code}")
```

Aetna (AET) Q4 2013 — top 5 holders:

	filerName	shares_M
Capital Research Global Investors		25.0
	State Street Corp	22.9
	WELLINGTON MANAGEMENT CO LLP	18.7
	VANGUARD GROUP INC	17.3
	FMR LLC	13.2

The endpoint works for historical CIKs — we just need to find the remaining 57.

The 57 we still can't find are all acquired companies where the old ticker no longer appears in SEC's active records. Examples: CELG (Celgene, acq. BMS 2019), YHOO (Yahoo, acq. Verizon 2017), XLNX (Xilinx, acq. AMD 2022).

Does QK maintain a historical ticker → CIK mapping for companies no longer publicly traded?

5.4 — Is pricing flat or proportional to data returned? (Resolved, its flat even in 2023)

This determines whether our 2023–2025 quarters cost 15 credits/firm or ~49 credits/firm. The cell pulls the same firm (Apple) for two quarters — 2013 (low investor count) and 2023 (high investor count) — and measures the credit difference.

```
In [24]: def pull_and_measure(yyyyqq):
    before = qk.account().usage
    raw = qk.holders(qk.ticker("AAPL"), yyyyqq=yyyyqq).to_df()
    after = qk.account().usage
    n = len(raw.query("putCall.isna() and shrsOrPrnAmt > 0", engine=
    return {"quarter": str(yyyyqq), "investors": n, "credits": after - be

r2013 = pull_and_measure(201304)
r2023 = pull_and_measure(202304)

result = pd.DataFrame([r2013, r2023])
result["credits_per_investor"] = (result["credits"] / result["investors"])

print(result.to_string(index=False))
print()

ratio_inv = r2023["investors"] / r2013["investors"]
ratio_cost = r2023["credits"] / r2013["credits"] if r2013["credits"] >
print(f"Investor count ratio (2023/2013): {ratio_inv:.2f}x")
print(f"Credit cost ratio (2023/2013): {ratio_cost:.2f}x")
if abs(ratio_inv - ratio_cost) < 1.0:
    print("→ Cost scales proportionally with investor count")
else:
    print("→ Cost appears flat per call regardless of investor count")
```

quarter	investors	credits	credits_per_investor
201304	1920	20	0.01
202304	4929	20	0.004

Investor count ratio (2023/2013): 2.57x
Credit cost ratio (2023/2013): 1.00x
→ Cost appears flat per call regardless of investor count

```
In [ ]: qk.account()
```

```
Out[ ]: QUANTkiosk Account: ( Saturday, 16-May-26 06:40:58 UTC )
```

Daily Quota: 10000 (Hard Quota: 10000)
Daily Usage: 220

Daily quota resets in: 17h 19m

Visit <https://quantkiosk.com/account> to change your plan or explore other rings.

```
In [ ]:
```

